



Decomposição e ocultação de informações

1

- Princípio da separação de conceitos

No desenvolvimento de software, um dos princípios fundamentais é o da separação de conceitos (em inglês, Separation of Concerns, ou SOC). Esse princípio é uma prática essencial para garantir que os sistemas sejam bem organizados e fáceis de manter. Ele consiste em dividir o software em partes menores, cada uma responsável por tratar uma preocupação específica.

Conceito de Separação de Conceitos

O objetivo da separação de conceitos é garantir que cada módulo ou componente do software se concentre em uma única tarefa ou função. Isso melhora a coesão interna dos módulos, ou seja, garante que os elementos presentes em um módulo estejam intimamente relacionados. Quando uma função ou dado não se enquadra nesse contexto, ele deve ser realocado para outro módulo mais adequado.

Ao separar adequadamente os conceitos, conseguimos criar um sistema modular. A modularidade facilita tanto a construção quanto a manutenção do software, já que os diferentes módulos podem ser modificados sem impactar o funcionamento de outros.

Benefícios da Separação de Conceitos

Modularidade e Manutenção - Um sistema modular é mais fácil de entender e manter. Com módulos bem definidos e focados em suas

respectivas responsabilidades, é possível fazer atualizações ou correções em uma parte do sistema sem influenciar outras. Isso não apenas reduz o risco de erros, mas também acelera o processo de desenvolvimento.

Reutilização - Outro benefício é a possibilidade de reutilização de módulos. Quando os conceitos estão bem separados, os módulos podem ser reutilizados em diferentes partes do sistema ou até em outros projetos. Essa reutilização otimiza o esforço e o tempo investidos no desenvolvimento.

Compreensibilidade - Sistemas com conceitos separados são mais compreensíveis. Desenvolvedores podem focar em entender apenas o módulo com o qual estão trabalhando, sem a necessidade de dominar todo o sistema. Isso torna o processo de manutenção mais ágil e eficiente.

Testabilidade - A separação de conceitos também melhora a testabilidade do software. Com módulos coesos, é mais fácil escrever testes específicos para cada funcionalidade, tornando o processo de verificação e correção de erros mais simples e confiável.

Exemplo Prático

Imagine a criação de uma classe “Smartphone” que contenha tanto funções relacionadas a chamadas telefônicas quanto funções de câmera. Inicialmente, pode parecer conveniente combinar essas funções, mas isso resultaria em baixa coesão. Os comportamentos da câmera e do telefone não têm relação direta e, ao mantê-los juntos, dificultamos a manutenção e a extensão das funcionalidades.

Aplicando o princípio da separação de conceitos, seria mais adequado criar uma classe para o telefone e outra para a câmera. Cada uma delas cuidaria apenas de suas respectivas funcionalidades, promovendo um design mais

claro e modular. Dessa forma, seria possível modificar ou substituir uma funcionalidade sem afetar a outra.

2 - Decomposição

A decomposição é um princípio fundamental no design de software, essencial para lidar com a complexidade de sistemas. Esse conceito envolve dividir um sistema ou problema complexo em componentes menores e mais gerenciáveis. A partir dessa divisão, cada parte pode ser desenvolvida, compreendida e mantida de maneira independente, facilitando o processo de desenvolvimento e manutenção.

O que é Decomposição?

Quando aplicamos a decomposição, estamos essencialmente quebrando um grande problema em partes menores, que são mais fáceis de entender e tratar. Em um sistema com pouca decomposição, onde tudo está concentrado em uma única classe ou módulo, tendemos a enfrentar dificuldades como baixa coesão e alto acoplamento, o que complica a manutenção e a adição de novas funcionalidades. Já em um sistema devidamente decomposto, cada módulo se foca em uma responsabilidade específica, permitindo uma melhor organização e evolução do software.

Vantagens da Decomposição

A principal vantagem da decomposição é a capacidade de isolar problemas. Isso significa que podemos concentrar nossos esforços em componentes específicos do sistema, seja para manutenção, aprimoramento ou até mesmo para construção inicial. Isso facilita não apenas o desenvolvimento, mas também a evolução contínua do sistema, pois alterações em uma parte do código não afetam diretamente outras partes.

Exemplo Prático

Vejamos um exemplo prático para ilustrar. Suponha uma classe chamada “Biblioteca”. Essa classe contém os seguintes atributos:

- Uma lista de objetos do tipo “Livro”.
- Uma matriz de objetos do tipo “Usuário”.
- Uma matriz de objetos do tipo “Empréstimo”.

Além disso, a classe possui métodos como adicionar livros, adicionar usuários e registrar empréstimos. Embora tudo esteja relacionado à biblioteca, notamos que estamos lidando com conceitos distintos: livros, usuários e empréstimos. Essa mistura de responsabilidades em uma única classe torna o sistema menos coeso.

Aplicando o princípio da decomposição, podemos dividir essa classe em três classes menores, cada uma com uma responsabilidade bem definida:

Classe Livro: Responsável por armazenar as informações dos livros, como título, autor e ISBN. Essa classe teria métodos específicos para manipular esses atributos (os conhecidos getters e setters), garantindo que sua única função seja a manutenção das informações sobre os livros.

Classe Usuário: Focada nas informações dos usuários, como nome e ID. Assim como a classe Livro, essa classe também teria métodos para manipular esses atributos de forma isolada, sem misturar com outras funcionalidades.

Classe Empréstimo: Esta classe lidaria exclusivamente com os dados dos empréstimos, como o livro emprestado, o usuário que fez o empréstimo, a data de empréstimo e a data de devolução. Aqui, a responsabilidade é gerenciar o ciclo de vida dos empréstimos.

Com essa abordagem, transformamos um problema complexo em problemas menores e mais específicos, com conceitos melhor definidos e responsabilidades claramente separadas. A decomposição, portanto, permite que o software seja mais organizado, fácil de manter e escalável. Ao isolar as responsabilidades em diferentes classes, conseguimos um design mais robusto e eficiente.

3 - Ocultação de informações

A ocultação de informações é um princípio fundamental no design de software, que estabelece que cada módulo ou componente de um sistema deve ter acesso apenas ao que é necessário para cumprir sua função. Ao aplicar esse conceito, buscamos reduzir a dependência entre os módulos, promovendo um design mais coeso e flexível.

Conceito de Ocultação de Informações

A ideia principal da ocultação de informações é impedir que os módulos de um sistema conheçam detalhes desnecessários de outros módulos, minimizando o impacto de futuras alterações. Cada módulo deve se concentrar em suas próprias responsabilidades e expor somente o que é essencial para sua interação com o restante do sistema. Esse processo é alcançado por meio de interfaces, que oferecem aos outros módulos acesso limitado, garantindo que os detalhes internos permaneçam ocultos.

O princípio vai além da simples proteção de dados sensíveis. Ele se aplica também a qualquer informação ou detalhe que possa ser alterado ao longo do tempo, como implementações específicas de algoritmos ou estruturas de dados. A ocultação de informações protege essas partes do sistema, evitando que alterações internas causem problemas em outros módulos.

Benefícios da Ocultação de Informações

A ocultação de informações oferece uma série de vantagens no design de software. Um dos principais benefícios é a independência no desenvolvimento dos módulos. Cada equipe ou desenvolvedor pode trabalhar em uma parte específica do sistema sem a necessidade de entender como os outros módulos funcionam internamente. Isso garante maior foco e eficiência no desenvolvimento, além de reduzir a chance de erros ou interferências indesejadas.

Outro benefício significativo é a flexibilidade para futuras modificações. Quando um módulo está corretamente encapsulado e suas informações são devidamente ocultadas, é possível reescrevê-lo ou otimizá-lo sem que essas mudanças afetem o comportamento dos outros módulos. Desde que a interface pública permaneça a mesma, o sistema continuará funcionando como esperado.

Relação com Encapsulamento

A ocultação de informações está diretamente relacionada ao conceito de encapsulamento na programação orientada a objetos. O encapsulamento garante que os detalhes internos de uma classe ou módulo estejam protegidos e que o acesso a esses detalhes ocorra apenas por meio de métodos ou interfaces públicas. Modificadores de acesso, como “privado”, “protegido” e “público”, são utilizados para definir o nível de visibilidade de atributos e métodos, permitindo o controle sobre o que deve ou não ser acessível de fora do módulo.

Ao encapsular corretamente as funcionalidades e dados de um módulo, reforçamos a ideia de modularidade e isolamento, princípios essenciais para um bom design de software.

Como Avaliar o Que Deve Ser Ocultado

Uma forma prática de determinar o que deve ser ocultado é fazer a seguinte pergunta: Isso pode mudar? Se a resposta for positiva, significa

que esses detalhes devem ser ocultados, pois há a possibilidade de alteração ao longo do tempo. Informações que podem sofrer mudanças, como implementações internas e detalhes de algoritmos, são bons exemplos de elementos que devem ser protegidos por meio da ocultação de informações.

Por outro lado, premissas fixas, como comportamentos esperados ou requisitos de hardware, devem ser expostas por meio de interfaces claras, uma vez que não se espera que mudem com frequência.

A aplicação correta da ocultação de informações resulta em sistemas mais modulares e fáceis de manter. Ao proteger os detalhes internos de cada módulo, garantimos que eles possam ser alterados ou otimizados sem impactar o restante do sistema. Com isso, o design de software se torna mais robusto e flexível, atendendo às demandas de manutenção e evolução de longo prazo.

4 - Ocultação de informações nos diagramas do Astah

Nesta etapa, vamos explorar a aplicação do princípio de ocultação de informações utilizando o software Astah. O objetivo é mostrar, de forma prática, como manipular modificadores de acesso em diagramas de classes e gerar o código-fonte a partir desses diagramas. Além disso, veremos como realizar o processo inverso, importando o código gerado para o Astah e verificando a coerência entre o diagrama e o código.

Criando um Diagrama de Classes no Astah

Para começar, criaremos um diagrama de classes no Astah. A ferramenta permite definir atributos e métodos para cada classe, além de configurar a visibilidade de cada elemento. Vamos tomar como exemplo a classe “Livro”, e, a partir dela, veremos como aplicar os diferentes modificadores de acesso.

Atributos e Modificadores de Acesso:

→ Primeiro, criamos a classe “Livro” e adicionamos o atributo título, do tipo String. Por padrão, o Astah define esse atributo como privado, indicado por um sinal de menos (-). No entanto, podemos alterar essa visibilidade para público, o que é representado pelo sinal de mais (+).

→ Em seguida, criamos o atributo autor e definimos sua visibilidade como protegida. Esse tipo de acesso, indicado por um jogo da velha (#), permite que o atributo seja acessado pela própria classe e por suas subclasses.

→ Também adicionamos o atributo quantidade, do tipo int, que permanecerá com visibilidade privada.

→ Por fim, o atributo preço será configurado com visibilidade de pacote (package), representado pelo símbolo til (~), o que significa que ele pode ser acessado por outras classes dentro do mesmo pacote.

Operações (Métodos):

→ Para ilustrar o uso de operações, adicionamos o método getTítulo, que retorna uma String. Esse método será público, permitindo que outras classes o utilizem para acessar o título do livro.

Gerando o Código-Fonte Java

→ Uma das funcionalidades úteis do Astah é a capacidade de gerar código-fonte a partir dos diagramas. Vamos exportar nossa classe “Livro” para Java:

→ Após criar a classe com seus atributos e métodos, acessamos o menu “Tools” e selecionamos a opção para gerar o código em Java.

→ O Astah exportará a classe, gerando um arquivo com os atributos e métodos conforme foram definidos no diagrama. No código gerado, os

modificadores de acesso seguirão as configurações feitas no Astah, como o atributo título sendo público e o atributo autor protegido.

Verificando a Ocultação de Informações no Código Java

→ Ao abrir o arquivo gerado em um editor de texto, podemos verificar se os modificadores de acesso foram aplicados corretamente. Por exemplo:

→ O atributo título aparecerá como public, conforme configurado no diagrama.

→ O atributo autor estará marcado como protected, e o atributo quantidade como private.

→ Para o atributo preço, que foi definido como pacote, notamos que, em Java, a ausência de um modificador explícito significa que ele tem visibilidade de pacote, o que confirma a coerência entre o diagrama e o código gerado.

Fazendo o Processo Inverso: Importando o Código para o Diagrama

Outro recurso do Astah é a capacidade de importar código-fonte para gerar diagramas. Vamos modificar o código Java da classe “Livro” diretamente, alterando a visibilidade de alguns atributos e adicionando novos:

→ Mudamos os atributos título e autor para privado e adicionamos um novo atributo, resumo, também privado.

→ Ao importar o código novamente para o Astah, o diagrama de classes será atualizado automaticamente para refletir essas mudanças, mostrando todos os atributos como privados, incluindo o novo atributo resumo.

Conclusão

A utilização de ferramentas de modelagem como o Astah facilita a aplicação dos princípios de ocultação de informações ao design de software. Modificar a visibilidade de atributos e métodos diretamente no diagrama e verificar a correspondência no código-fonte ajuda a garantir um sistema bem encapsulado e de fácil manutenção. Além disso, o Astah permite tanto criar diagramas a partir do código quanto gerar código a partir dos diagramas, proporcionando flexibilidade no processo de desenvolvimento.

Conteúdo Bônus

Para aprofundar seu conhecimento sobre decomposição e ocultação de informações no design de software, recomendo o vídeo 'Engenharia de Software: Reúso de Software' disponível gratuitamente no YouTube, no canal: UNIVESP. Embora o tema principal seja o reúso, ele aborda conceitos relacionados à decomposição de sistemas e boas práticas de design, como modularidade e encapsulamento, que estão diretamente conectados ao conceito de ocultação de informações. Esse conteúdo é bastante útil para entender como módulos podem ser estruturados para facilitar a reutilização e ocultação de detalhes de implementação.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019.

Título da Prática: criação de classes no Astah

Objetivos: criar um diagrama de classes no Astah explorando o conceito de SRP e ocultação de informações

Materiais, Métodos e Ferramentas: ferramenta de modelagem UML Astah UML

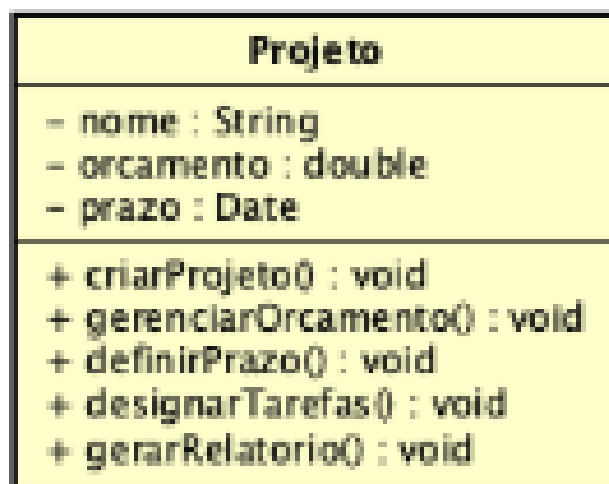
Roteiro

1º PASSO) leia atentamente o texto a seguir:

O Princípio da Responsabilidade Única (SRP) e a ocultação de informações são conceitos fundamentais na engenharia de software que promovem a manutenção e a robustez do código. O SRP diz que uma classe deve ter apenas uma responsabilidade ou razão para mudar, o que facilita a compreensão e modificação do código, pois cada classe é focada em uma única tarefa. Já a ocultação de informações envolve manter os detalhes internos de uma classe escondidos de outras classes, expondo apenas o que é necessário através de interfaces públicas. Isso minimiza a dependência entre diferentes partes do sistema, tornando-o mais modular e menos suscetível a erros quando mudanças são necessárias. Juntos, SRP e ocultação de informações ajudam a criar sistemas de software mais coesos e menos acoplados, permitindo uma evolução contínua e sustentável do código.

2º PASSO) analise o diagrama de classes da Figura 1

Figura 1 – diagrama de classes



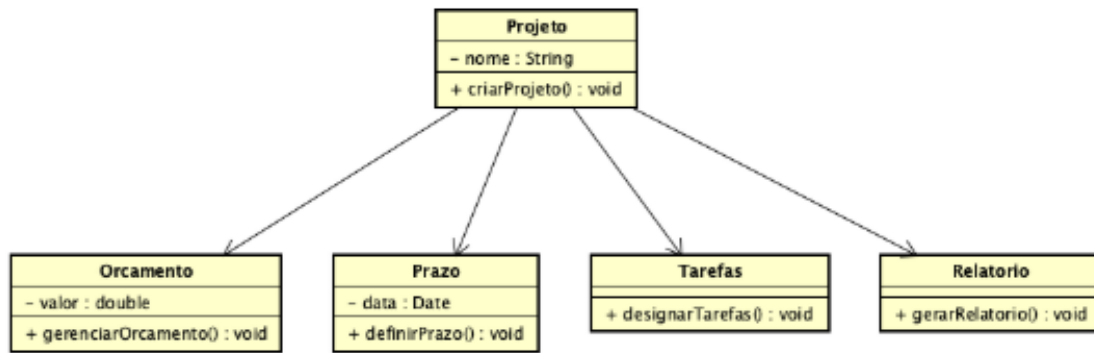
A) Gere um novo diagrama de classes a partir da Figura 1 aplicando o conceito de responsabilidade única (SRP).

B) Aplique o princípio da ocultação de informações no diagrama gerado.

GABARITO:

A) O diagrama deve ser semelhante ao da Figura 2.

Figura 2 – diagrama de classes



B) Os atributos devem ser privados e os métodos públicos.

Ir para exercício